Algoritmi modulo II

- Interval scheduling

  Definizione:

Dati in input n intervalli: $I_1, \ldots, I_n$ I_i ha tempo di inizio S_i e tempo di fine f_i Due intervallo sono compatibili se non si sovrappongono Trovare il sottoinsieme di intervalli compatibili con cardinalità massima

  - Greedy algo

    Ordina gli intervali per tempo di fine f

    Complessità: O(n log n)

EARLIEST-FINISH-TIME-FIRST $(n, s_1, s_2, \ldots, s_n, f_1, f_2, \ldots, f_n)$

SORT jobs by finish times and renumber so that $f_1 \leq f_2 \leq \ldots \leq f_n$.

$S \leftarrow \varnothing$. ← set of jobs selected

FOR $j = 1$ TO n

  IF (job j is compatible with S)

    $S \leftarrow S \cup \{ j \}$.

RETURN S.

correttezza:

Sia $i_1, \ldots, i_k$ il set di intervalli selezionati dall'algo (ordinati per finish time)

Sia $j_1, \ldots, j_m$ il set di intervalli selezionato dall'ottimo (ordinati per f. time)

Con $f(i_r)$ intendiamo il tempo di fine del intervallo i_r

Lemma: per ogni r = 1,2,..,k abbiamo $f(i_r) <= f(j_r)$ (vedi dimostrazione)

Assumiamo che l'algo non sia ottimo

Allora m>k

Quindi la soluzione ottima ha almeno 1 intervallo in più

Ma per il lemma sappiamo che l'intervallo dell'ottimo corrispondente all'ultimo intervallo del greedy finisce o insieme o dopo l'ultimo intervallo del greedy

Perciò se ci fosse un altro intervallo compatibile che inizia dopo il greedy l'avrebbe scelto.

- Interval partitioning

Definizione:

Date in input un insieme di n intervalli (lezioni): $j_1,\ldots,j_n$
Lezione j inizia a s_j e finisce a f_j
Trovare il minimo numero di classi per schedulare tutte le lezioni affinché non ci siano due lezioni contemporaneamente nella stessa classe(ovvero le lezioni nella stessa classe devono essere compatibili)

- Greedy algo

Ordinare per tempo di inizio s_i

EARLIEST-START-TIME-FIRST $(n, s_1, s_2, \ldots, s_n, f_1, f_2, \ldots, f_n)$

SORT lectures by start times and renumber so that $s_1 \leq s_2 \leq \ldots \leq s_n$.

$d \leftarrow 0$. ← number of allocated classrooms

FOR $j = 1$ TO n

  IF (lecture j is compatible with some classroom)

    Schedule lecture j in any such classroom k.

  ELSE

    Allocate a new classroom $d + 1$.

    Schedule lecture j in classroom $d + 1$.

    $d \leftarrow d + 1$.

RETURN schedule.

Complessità O(n log n):

Dim.

- Ordina per tempo di inizio O(n log n)
- Salva le classi in una coda con priorità (key= finish time dell'ultimo job inserito)
  - Per allocare una nuova classe, INSERT nella coda priorità
  - Per schedulare una lezione, INCREASE-KEY della classe scelta
  - FIND-MIN ogni iterazione per scegliere la classe in cui schedulare la lezione
- # totale delle operazioni sulla coda priorità O(n), ognuna costa O(log n).

Def: la **depth** di un insieme di intervalli è il numero massimo di intervalli che contengono lo stesso punto

Osservazione chiave:

Numero di classi necessarie >= depth, nell'ottimo si eguagliano

L'algo è ottimo :

- Sia d = # di classi che l'algo alloca
- La classe d viene aperta poichè dobbiamo schedulare una lezione j che è incompatibile con la lezione d-1 di ogni altra classe
- Perciò queste d-1 lezioni finiscono tutte dopo s_j
- Poiché abbiamo ordinato per tempo di inizio tutte queste d-1 lezioni iniziano non più tardi di s_j
- Perciò avremo d lezioni in corso nel momento s_j + epslon
- Per osservazione chiave tutti gli schedulatori usano almeno d classi

- Union-find:

  Definizione:

  Mantenere una **collezione di insiemi disgiunti** contenenti elementi distinti durante l'esecuzione di una sequenza di operazioni del seguente tipo:
  - **Makeset(x)** = crea il nuovo insieme x = {x} di nome x
  - **Union(A,B)** = unisce gli insiemi A, B in un unico insieme di nome A (distrugge quelli vecchi)
  - **Find(x)** = restituisce il nome dell'insieme che contiene l'elemento x (si suppone di accedere direttamente all'elemento x)

  - QuickFind:

    Usiamo una foresta di alberi di altezza 1 per rappresentare gli insiemi disgiunti. In ogni albero:
    - Radice = nome dell'insieme
    - Foglie = elementi (**incluso** l'elemento rappresentativo, il cui valore è nella radice e da il nome all'insieme stesso)

Operazioni:

Makeset(elem e)	**T(n) = O(1)**
Crea un albero composto da due nodi: una radice e una foglia. Memorizza e sia nella foglia che nella radice come nome dell'albero	

Union(name a, name b)	**T(n) = O(n)**
Sostituisce tutti i puntatori delle foglie di B alla radice di B con puntatori alla radice di A. cancella la vecchia radice di B.	

Find(elem e) -- name	**T(n) = O(1)**
Accede alla foglia x corrispondente all'elemento e. da tale nodo segue il puntatore al padre che è la radice dell'albero e restituisce il nome memorizzato in tale radice	

- QuickFind con euristica union-by-size

Union by size:

Nell'unione degli insiemi A e B attacchiamo gli elementi dell'insieme con cardinalità minore a quello di cardinalità maggiore e se necessario modifichiamo la radice dell'albero ottenuto

Ogni insieme mantiene esplicitamente anche la propria size (numero di elementi)

Union(name a, name b) =	**T_{am} = O(log n)**

T_{am}= tempo per operazione ammortizzato sull'intera sequenza di unioni

Un nodo può **cambiare padre al più log n volte** in una sequenza di n-1 union poiché la cardinalità del nuovo insieme sarà almeno il doppio di quella dell'insieme precedente.
Perciò il tempo speso sulla sequenza di n-1 union per ogni singolo nodo è O(log n) ergo l'intera sequenza è: O(n log n)

In una sequenza di **m find, n makeset, n-1 union** il tempo richiesto dall'intera sequenza è:

O(m + n log n)

- QuickUnion

Usiamo una foresta di alberi di altezza anche maggiore di 1 per rappresentare gli insiemi disgiunti. In ogni albero:

- Radice = elemento rappresentativo dell'insieme
- Rimanenti nodi = altri elementi(**escluso** l'elemento nella radice)

Operazioni:

Makeset(x) = O(1)
Union(A,B) = O(1)
Find(x) = O(n)

- QuickUnion con euristica union-by-size

Union by size:
Nell'unione degli insiemi A e B, rendiamo la radice dell'albero con meno nodi figlia della radice dell'albero con più nodi e cambiamo il nome se necessario

L'operazione **find** richiede **tempo O(log n)**

Un intera sequenza di **n makeset, n-1 union, m find** richiede:

O(n+ m log n)

- QuickUnion euristica union-by-size e compressione dei cammini

  Compressione dei cammini:

  Quando eseguo find e attraverso il cammino da x alla radice, comprimo il cammino, ovvero rendo tutti i nodi del cammino figli della radice

  Teorema:

Usando in QuickUnion le euristiche union by size e compressione dei cammini, una qualsiasi sequenza di **n makeset, n-1 union, m find** hanno un costo di O(n+m **α** (n+m, n))

  α(x, y): funzione inversa della funzione di Ackermann

  α(m,n) < = 4 per valori raggionevoli di n

- Minimum Spanning Tree

  Definizione:

Dato un grafo connesso e non orientato G=(V,E), pesato sugli archi. Trovare un sottografo connesso, che tocca tutti i nodi, di costo totale minimo

  Proprietà:

  Se G ha pesi distinti allora esiste un unico MST

  - Cycle and Cut propriety

    Cycle: set di archi della seguente forma: a-b,b-c,…,y-z,z-a

    Cut: sottoinsieme di nodi S (a volte definito come partizione V in S e V\S)

Cutset: il cutset D associato al tagli S è il sottinsieme degli archi con esattamente un estremo in S

Cycle-cut intersection:

L'intersezione tra un cutset e un ciclo ha cardinalità pari

Cut propriety:

Sia S un qualunque taglio e sia **e** l'arco del cutset dal costo minimo. Allora esiste un MST che contiene **e**

Vedi dim sul quaderno

Cycle propriety:

Sia C un qualsiasi ciclo e sia **f** l'arco appartenente al ciclo di peso massimo. Allora esiste un MST che non contiene **f**

Vedi dim sul quaderno

- Kruskal's algo

  Inizia con T= \0 (insieme vuoto), considera gli archi in ordine crescente di peso, inserisci l'arco e a T se non genera un ciclo

  Un implementazione efficente di questo algoritmo utilizza la UnionFind

```
algorithm Kruskal (graph G=(V,E,c))
    UnionFind UF
    T=∅
    sort the edges in ascending order of costs
    for each vertex v do UF.makeset(v)
    for each edge (x,y) in order do
        T_x=UF.find(x)
        T_y=UF.find(y)
        if T_x≠ T_y then
          UF.union(T_x, T_y)
          add edge (x,y) to T
    return T
```

Correttezza: *guarda analisi sul quaderno e sulle slide*

Complessità:

O(m log n)

- Reverse-delete algo

  Si comporta in modo complementare all'algo di kruskal: Parte con T= G, cosnidera gli archi in ordine decrescente di peso, l'arco e viene rimosso solo se rimuovendolo il grafo resta connesso

- Prim's algo

  Inizia da un nodo radice s e fa crescere un albero T in modo greedy da s verso l'esterno, ad ogni passo aggiunge a T l'arco meno costoso che ha esattamente un estremo in T

  Correttezza: conseguenza diretta della cut proriety, usata n-1 volte

  Un implementazione veloce:
  - Mantiene l'insieme dei nodi esplorati S
  - Usa una coda priorità per mantenere i nodi inesplorati

- Per ogni nodo inesplorato la pririta è l'attachment cost a[v] = costo dell'arco meno costoso incidente a v che ha l'altro estremo in S

```
Prim(G, s) {
   foreach (v ∈ V) a[v] ← ∞
   a[s] ← 0
   Initialize an empty priority queue Q
   foreach (v ∈ V) insert v onto Q with priority a[v]
   Initialize set of explored nodes S ← ϕ
   Initialize T to the tree containing only s.

   while (Q is not empty) {
      u ← delete min element from Q
      S ← S ∪ { u }
      foreach (edge e = (u, v) incident to u)
          if ((v ∉ S) and (c_e < a[v]))
             make u parent of v in T
             decrease priority a[v] to c_e
   return T
}
```

Complessità:

O(m + n log n)
Coda di priorità implementata con heap di Fibonacci

- Clustering

Definizione:

Dato un insieme U di n elementi etichettati $p_1,\ldots,p_n$ e un intero k, dividere gli ogetti in k sottoinsiemi non vuoti affinché lo spacing sia massimo

Spacing = distanza minima tra una qualsiasi coppia di punti in cluster diversi
La distanza viene espressa da una funzione

  - Greedy algo (Kruskal's algo)
    La procedura è corrispondente all'algo di Kruskal, solo che poi bisogna togliere i k-1 archi più pesanti dal grafo

Programmazione dinamica

- Insieme Indipendente di peso massimo (su grafi a cammino)

Definizione:

Dato un cammino **G** di **n** nodi. Ogni nodo $\mathbf{v_i}$ ha **peso** $\mathbf{w_i}$.
Trovare un insieme indipendente di **peso massimo** ovvero un insieme **S** tale che:

(i) **S** è un insieme indipendente

(ii) w(**S**) = somma dei pesi dei nodi in **S**, è più grande possibile

Un **insieme indipendente** di **G** è un sottoinsieme di nodi che non contiene due nodi adiacenti, ovvero per ogni coppia di nodi del insieme i due nodi non sono collegati da un arco.

G_j : sottocammino composto dai primi **j** vertici di **G**

Sottoproblema j : calcolare il peso del migliro insieme indipendente per **G_j**.

OPT[j] : valore della soluzione sottoproblema j, ovvero peso dell'insieme indipendente di peso massimo di **G_j.**

OPT[1] = w_1 ; OPT[2] = max{ w_1 , w_2}

OPT[j] = max{ OPT[j-1], w_j+OPT[j-2]

Algo:

```
1.  OPT[1]=w1; OPT[2]= max {w1, w2}
2.  for j=3 to n do
3.      OPT[j]= max {OPT[j-1], wj+OPT[j-2]}
4.  return OPT[n]
```

$T(n)=\Theta(n)$

Oss: l'algoritmo fornisce il valore della soluzione ma non la soluzione

Algo per ricostruire la soluzione:

```
S*=∅; j=n;
while j≥3 do
    if OPT[j-1] ≥ w_j+OPT[j-2]
    then j=j-1;
    else S*=S*∪{ v_j }; j=j-2;
if j=2 e w_2>w_1 then S*=S*∪{ v_2 }
                 else S*=S*∪{ v_1 };
return S*
```

complessità temporale?

$$\boxed{T(n)=\Theta(n)}$$

- Weighted Interval Scheduling

Definizione:

- Dati **n** intervalli chiamati job $\mathbf{j_1,\ldots,j_n}$
- Job **j** inizia a $\mathbf{s_j}$ e finisce a $\mathbf{f_j}$, e ha peso $\mathbf{w_j} > 0$.
- Due job **sono compatibili** se non si sovrappongono
- Trovare il sottoinsieme di job compatibili di peso massimo

Convenzione: ordino i job per finish time in modo crescente

Def: p(j) = più grande indice i < j tale che job i sia compatibile con job j

Def sottoproblema

OPT[j] = peso massimo di un qualsiasi sottoinsieme di job compatibili costituito solo dai job 1,2,..,j

La soluzione cercata si trova in OPT[n].

Caso 1. OPT[j] non seleziona j:

- Allora l'ottimo di j deriva dall'ottimo dei primi j-1esimi job

Caso 2. OPT[j] seleziona j:

- Colleziona il costo di w_j
- Non può usare i job incompatibili { p(j) +1, p(j)+2, ..., j-1}.
- Deve contenere una soluzione ottima per i job compatibili rimanenti 1, 2 , .., p(j)

Bellman equation. $$OPT(j) = \begin{cases} 0 & \text{if } j = 0 \\ \max\{OPT(j-1),\ w_j + OPT(p(j))\} & \text{if } j > 0 \end{cases}$$

Algo:

BOTTOM-UP$(n, s_1, \ldots, s_n, f_1, \ldots, f_n, w_1, \ldots, w_n)$

Sort jobs by finish time and renumber so that $f_1 \leq f_2 \leq \ldots \leq f_n$.

Compute $p[1], p[2], \ldots, p[n]$ via binary search.

$M[0] \leftarrow 0$.

previously computed values

FOR $j = 1$ TO n

$\quad M[j] \leftarrow \max\{M[j-1],\ w_j + M[p[j]]\}$.

RETURN $M[n]$.

Complessità:

O(n log n)

- Longest Increasing Subsequence

Definizione:

Data una sequenza di n oggetti $i_1, \ldots, i_n$ L'ogetto i_i ha valore s_i

Trovare una sottoseguenza di cardinalità massima affinché per ogni coppia di oggetti i,j con i < j appartenenti alla sottoseguenza vale che $S_i < S_j$

Def sotto problema

OPT[j] = lunghezza della longest increasing subsequence di S[1] , … , S[i] che termina con S[i]

La soluzione cercata è il $\max_{i=1,2,..,n}$ OPT[i]

Il caso base è OPT[1]

Algo:

LIS(S[1:n])

OPT[1]=1

FOR $i = 2$ TO n

$$\text{OPT}[i] = 1 + \max\left\{0, \max_{\substack{j=1,2,\ldots,i-1 \\ \text{st } S[j]<S[i]}} \text{OPT}[j]\right\}$$

RETURN max_i OPT[i].

Complessità:

$O(n^2)$

- House coloring problem

HOUSE COLORING PROBLEM

Goal. Paint a row of n houses red, green, or blue so that

- No two adjacent houses have the same color.
- Minimize total cost, where $cost(i, color)$ is cost to paint i given color.

	A	B	C	D	E	F
	7	6	7	8	9	20
	3	8	9	22	12	8
	16	10	4	2	5	7

cost to paint house i the given color

HOUSE COLORING PROBLEM

Subproblems.

- $R[i]$ = min cost to paint houses $1, \ldots, i$ with i red.
- $G[i]$ = min cost to paint houses $1, \ldots, i$ with i green.
- $B[i]$ = min cost to paint houses $1, \ldots, i$ with i blue.
- Optimal cost = $\min \{ R[n], G[n], B[n] \}$.

Dynamic programming equation.

- $R[i] = cost(i, red) + \min \{ B[i-1], G[i-1] \}$
- $G[i] = cost(i, green) + \min \{ R[i-1], B[i-1] \}$
- $B[i] = cost(i, blue) + \min \{ R[i-1], G[i-1] \}$

overlapping subproblems

Running time. $O(n)$.

- Segmented Least Squares

Least squares definizione:

Dati n punti sul piano: $(x_1,y_1),\ldots,(x_n,y_n)$
Trovare una retta y= ax+b che minimizza la somma degli errori quadratici:

$$SSE = \sum_{i=1}^{n} (y_i - ax_i - b)^2$$

Solution. Calculus $\Rightarrow$ min error is achieved when

$$a = \frac{n\sum_i x_i y_i - (\sum_i x_i)(\sum_i y_i)}{n\sum_i x_i^2 - (\sum_i x_i)^2}, \quad b = \frac{\sum_i y_i - a\sum_i x_i}{n}$$

Segmented Least Squares definizione:

I punti si trovano approssimativamente su una sequenza di diversi segmenti
Dati n punti sul piano: $(x_1,y_1),\ldots,(x_n,y_n)$ con $x_1<x_2<\ldots<x_n$
Trova una sequenza di linee che minimizza f(x)

f(x) = E + c L per una qualche costante c>0, dove:

- E = somma delle somme degli errori quadratici di ogni segmento
- L = numero di segmenti

Def sottoproblemi:

- OPT[j] = costo minimo per i punti $p_1,\ldots,p_j$

- e_{ij} = SSE per i punti $p_i, p_{i+1}, \ldots, p_j$

Per calcolare OPT[j] :

- L'ultimo segmento utilizza i punti $p_i, p_{i+1}, \ldots, p_j$ per qualche i<j
- Costo = e_{ij} + c + OPT[j-1]

Bellman equation.

$$OPT(j) = \begin{cases} 0 & \text{if } j = 0 \\ \min\limits_{1 \le i \le j} \{ e_{ij} + c + OPT(i-1) \} & \text{if } j > 0 \end{cases}$$

SEGMENTED-LEAST-SQUARES$(n, p_1, \ldots, p_n, c)$

FOR j = 1 TO n

  FOR i = 1 TO j

    Compute the SSE e_{ij} for the points $p_i, p_{i+1}, \ldots, p_j$.

$M[0] \leftarrow 0.$

FOR $j = 1$ TO n

  $M[j] \leftarrow \min_{1 \le i \le j} \{ e_{ij} + c + M[i-1] \}.$

previously computed value

RETURN $M[n]$.

Precalcolando le sommatorie della foruma di sopra il tempo diventa O(n^2) altrimenti è O(n^3)

- Knapsack Problem

  Definizione:

  - Dati n elementi:
  - L'elemento i fornisce un valore v_i e pesa w_i

- Valore di un sottoinsieme di oggetti = somma dei valori dei singoli oggetti
- Lo zaino ha peso limite W
- Riempire lo zaino in modo tale da massimizzare il valore degli oggetti trasportati

- Algo pseudopolinomiale O(n W)

Definizione sottoproblemi:

OPT[i , w] = valore ottimo del knapsak problem con oggetti 1,.., i con peso limite w

Goal: OPT[n, W]

Caso 1. OPT[i, w] non seleziona i:

- OPT [i, w] seleziona i migliori {1,2,..,i-1} con peso limite w

Caso 2. OPT[i, w] seleziona i:

- Colleziona il valore v_i
- Nuovo peso limite = $w-w_i$
- OPT[i, w] seleziona i migliori {1,2,...i-1} con nuovo peso limite

Bellman equation.

$$OPT(i,w) = \begin{cases} 0 & \text{if } i = 0 \\ OPT(i-1,w) & \text{if } w_i > w \\ \max\{\, OPT(i-1,w),\ \ v_i + OPT(i-1, w-w_i) \,\} & \text{otherwise} \end{cases}$$

KNAPSACK(n, W, w_1, …, w_n, v_1, …, v_n)

FOR $w = 0$ TO W

$M[0, w] \leftarrow 0$.

FOR $i = 1$ TO n

FOR $w = 0$ TO W

IF $(w_i > w)$ $M[i, w] \leftarrow M[i-1, w]$.

ELSE $M[i, w] \leftarrow \max \{ M[i-1, w],\ v_i + M[i-1, w-w_i] \}$.

previously computed values

RETURN $M[n, W]$.

Complessità:

O (n W)

L'algoritmo è pseudo-polinomiale

- Sequence Alignment / edit distance

Def edit distance:

- Gap penalty δ; mismatch penalty α_{pq}.
- Cost = sum of gap and mismatch penalties.
- *L'edit distance è definita come l'allineamento di costo minimo*

Sequence Alignment definizione:

- Date 2 stringhe $x_1x_2...x_n$ e $y_1y_2...y_n$ trovare un allineamento di costo minimo
- Un allineamento M è un insieme di coppie ordinate x_i-y_i affinchè ogni carattere appaia al massimo in una coppia e senza incroci

- Il costo di un allineamento M è:

$$\text{cost}(M) = \underbrace{\sum_{(x_i, y_j) \in M} \alpha_{x_i y_j}}_{\text{mismatch}} + \underbrace{\sum_{i : x_i \text{ unmatched}} \delta + \sum_{j : y_j \text{ unmatched}} \delta}_{\text{gap}}$$

Definizione sottoproblemi:

OPT[i,j] = costo minimo di allineamento delle stringhe di prefisso $x_1x_2...x_i$ e $y_1y_2...y_i$

Goal. OPt[m,n]

Caso 1. OPT[i,j] metcha x_i-y_i

- Paga mismatch per x_i-y_i + costo minimo per allineare $x_1x_2...x_{i-1}$ e $y_1y_2...y_{i-1}$

Caso 2a. OPT[i,j] lascia x_i non metchata

- Paga gap per x_i + costo minimo per allineare $x_1x_2...x_{i-1}$ e $y_1y_2...y_i$

Caso 2b. OPT[i,j] lascia y_i non metchata

- Paga gap per y_i + costo minimo per allineare $x_1x_2...x_i$ e $y_1y_2...y_{i-1}$

Bellman equation.

$$OPT(i,j) = \begin{cases} j\delta & \text{if } i = 0 \\ i\delta & \text{if } j = 0 \\ \min \begin{cases} \alpha_{x_i y_j} + OPT(i-1, j-1) \\ \delta + OPT(i-1, j) \\ \delta + OPT(i, j-1) \end{cases} & \text{otherwise} \end{cases}$$

SEQUENCE-ALIGNMENT$(m, n, x_1, \ldots, x_m, y_1, \ldots, y_n, \delta, \alpha)$

FOR $i = 0$ TO m

$M[i, 0] \leftarrow i\delta.$

FOR $j = 0$ TO n

$M[0, j] \leftarrow j\delta.$

FOR i = 1 TO m

FOR j = 1 TO n

$$M[i,j] \leftarrow \min \{ \alpha_{x_i y_j} + M[i-1, j-1], \quad \delta + M[i-1, j], \quad \delta + M[i, j-1] \}.$$

already computed

RETURN $M[m, n]$.

Complessità:

O(m n)

- Shortest paths with negative weights

  Definizione:

  Dato un grafo diretto G = (V,E), con lunghezza degli archi arbitraria l_{vw} (può essere anche negativa) trovare un cammino minimo da un nodo sorgente s a un nodo destinazione t

  Lemma 1:

  Se qualche cammino v⇝t contiene un ciclo negativo allora non esiste un cammino minimo v⇝t

Lemma 2:

Se G non ha cicli negativi allora esiste un cammino minimo v⇝t che è un cammino semplie (ed ha ≤ n-1 archi)

- Dynamic programming algo

Definizione sottoproblemi:

OPT[i, v] = lunghezza del cammino minimo v⇝t che usa ≤ i archi

Goal. OPT[n-1, v] per ogni nodo v

Caso 1. cammino minimo v⇝t usa ≤ i archi

- OPT[i, v] = OPT[i-1, v]

Caso 2. cammino minimo v⇝t usa esattamente i archi

- Se (v,w) è il primo arco nel cammino minimo v⇝t comporta un costo di l_{vw}
- Poi seleziona il miglior cammino w⇝t che usa ≤ i-1 archi

Bellman equation.

$$OPT(i,v) = \begin{cases} 0 & \text{if } i = 0 \text{ and } v = t \\ \infty & \text{if } i = 0 \text{ and } v \neq t \\ \min\left\{ OPT(i-1,v),\ \min\limits_{(v,w)\in E} \{OPT(i-1,w) + \ell_{vw}\} \right\} & \text{if } i > 0 \end{cases}$$

SHORTEST-PATHS(V, E, ℓ, t)

FOREACH node $v \in V$:

$M[0, v] \leftarrow \infty$.

$M[0, t] \leftarrow 0$.

FOR i = 1 TO $n - 1$

FOREACH node $v \in V$:

$M[i, v] \leftarrow M[i-1, v]$.

FOREACH edge $(v, w) \in E$:

$M[i, v] \leftarrow \min \{ M[i, v],\ M[i-1, w] + \ell_{vw} \}$.

Complessità:

O(m n) tempo e O(n^2) spazio

- Bellman-Ford-Moore algo

Ottimizzazione dello spazio:

Manteniamo due array invece che una matrice:

- d[v] = lunghezza del cammino v⇝t più breve che abbiamo trovato finora.
- successor[v] = prossimo nodo nel cammino v⇝t

Ottimizzazione della performance:

Se d[w] non viene aggiornato nella passata i-1, non c'è motivo di considerare gli archi entranti in w nella passata i.

BELLMAN–FORD–MOORE(V, E, ℓ, t)

FOREACH node $v \in V$:

  $d[v] \leftarrow \infty$.

  $successor[v] \leftarrow null$.

$d[t] \leftarrow 0$.

FOR $i = 1$ TO $n - 1$

  FOREACH node $w \in V$:

    IF ($d[w]$ was updated in previous pass)

      FOREACH edge $(v, w) \in E$:

        IF ($d[v] > d[w] + \ell_{vw}$)

          $d[v] \leftarrow d[w] + \ell_{vw}$.

          $successor[v] \leftarrow w$.

  IF (no $d[\cdot]$ value changed in pass i) STOP.

pass i
$O(m)$ time

Lemma 3:

Per ogni nodo v : d[v] è la lunghezza di qualche cammino v⇝t

Lemma 4:

Per ogni nodo v : d[v] è monotona non crescente

Lemma 5:

Dopo la passata i, d[v] ≤ lunghezza del cammino minimo v⇝t che usa ≤ i archi

Teorema 2:

Assumendo che non ci siano cicli negativi, Bellman-Ford-Moore calcola la lunghezza del cammino minimo in **O(m n) tempo** e **O(n) spazio**

Lemma 6:

Ogni ciclo diretto W nel grafo dei successori è un ciclo negatico

pass n
$O(m)$ time

FOREACH edge $(v, w) \in E$
    IF $(d[v] > d[w] + \ell_{vw})$ THEN return "there is a negative cycle"

Complessità:

O(m n) tempo e O(n) spazio

- Max-flow and min-cut problems

Flow network definizione:

Una rete di flusso è una tupla G=(V,E,s,t,c):
- Grafo diretto (V,E) con una sorgente s e un pozzo t
- Capacità c(e) ≥ 0 per ogni arco e

Def st-cut:

Un st-cut è una partizione (A,B) di nodi con s ∈ A e t ∈ B

Def capacità di un st-cut:

La capacità di un st-cut è la somma delle capacità degli archi che vanno da A in B

Min-cut problem:

Trova un taglio di capacità minima

Def st-flow:

Un st-flow f è una funzione che soddisfa:

- Per ogni arco e ∈ E: $0 \leq f(e) \leq c(e)$ *[capacity]*
- Per ogni nodo v ∈ V-{s,t}: $\sum_{e \text{ in to } v} f(e) = \sum_{e \text{ out of } v} f(e)$ *[flow conservation]*

Def valore di un st-flow:

Il valore di un st flow è:

$$val(f) = \sum_{e \text{ out of } s} f(e) - \sum_{e \text{ in to } s} f(e)$$

Max-flow problem:

Trovare un flusso dal valore massimo

Residual network definizione:

Arco originale. e=(u,v) ∈ E

- Flow f(e)
- Capacity c(e)

Arco al contrario. $e^{reverse}$ = (v,u)

- "undo" flusso mandato

Residual capacity:

$$c_f(e) = \begin{cases} c(e) - f(e) & \text{if } e \in E \\ f(e^{\text{reverse}}) & \text{if } e^{\text{reverse}} \in E \end{cases}$$

Residual network. $G_f = (V, E_f, s, t, c_f)$

- $E_f = \{ e : f(e) < c(e)\} \cup \{e : f(e^{reverse}) > 0\}$
- **Proprietà chiave** : f' è un flusso in G_f se f+f' è un flusso in G

Augmenting path definizione:

Def augmenting path:

Un cammino aumentante è un cammino semplice s⇝t nella rete residua G_f

Def bottleneck capacity:

La capacità di bottleneck di un cammino aumentante P è la capacità minima residua di ogni arco in P

Proprietà chiave:

Sia f un flusso e P un cammino aumentante in G_f allora, dopo aver chiamato f' ← AUGMENT(f,c,P), l'f' risultante è un flusso e val(f') = val(f) + bottleneck(G_f,P)

AUGMENT(f, c, P)

$\delta \leftarrow$ bottleneck capacity of augmenting path P.

FOREACH edge $e \in P$:

  IF ($e \in E$) $f(e) \leftarrow f(e) + \delta$.

  ELSE $f(e^{\text{reverse}}) \leftarrow f(e^{\text{reverse}}) - \delta$.

RETURN f.

- Ford-Fulkerson algo
  - Inizia con f(e) = 0 per ogni arco e ∈ E
  - Trova un cammino s⇝t P nella rete residua G_f
  - Aumenta il flusso lungo il cammino P
  - Ripeti finché non rimani bloccato

FORD–FULKERSON(G)

FOREACH edge $e \in E$: $f(e) \leftarrow 0$.

$G_f \leftarrow$ residual network of G with respect to flow f.

WHILE (there exists an s⇝t path P in G_f)

  $f \leftarrow$ AUGMENT(f, c, P).

  Update G_f.

RETURN f.

augmenting path

Lemma del valore del flusso.

Sia f un flusso qualsiasi e sia (A,B) un taglio qulsiasi. Allora, il valore del flusso f è uguale al flusso netto attraversante il taglio (A,B)

$$val(f) = \sum_{e \text{ out of } A} f(e) - \sum_{e \text{ in to } A} f(e)$$

Dualità debole:

Sia f un qualsiasi flusso e sia (A,B) un qualsiasi taglio .
Allora, val(f) ≤ cap(A,B).

Corollario:

Sia f un qualsiasi flusso e (A,B) un qualsiasi tagllio.
se val(f) = cap(A,B), allora f è un max flow e (A,B) è un min cut

Max flow min cut theorem:

Il valore di un flusso massimo = capacità di un taglio minimo

Augmenting path theorem :

Un flow f è un max flow se non ha cammini aumentanti

Teorema:

Dato un qualsiasi max flow f, possiamo calcolare il min cut (A,B) in O(m) tempo

Analisi di Ford-Fulkerson *(capacità intere)*

Assumendo:

Che la capacità di ogni arco c(e) sia un intero tra 1 e C

Teorema:

Ford-Fulkerson termia dopo alpiù **val(f*) ≤ nC** cammini aumentanti , dove f* è il flusso massimo

Complessità:

O(m val(f*)) = O(m n C)

L'algo è pseudo-polinomiale

Algoritmi di approssimazione

Definizione:

Un α-approximation algorithm per un problema di ottimizzazione è un algoritmo polinomiale che per tutte le istanze del problema produce una soluzione il cui valore è compreso entro un fattore α rispetto al valore di una soluzione ottima

minimization problem:

- $\alpha \geq 1$

- Per ogni soluzione restituita x, cost(x) ≤ α OPT(x)

Maximization problem:

- α ≤ 1
- Per ogni soluzione restituita x, value(x) ≥ α OPT(x)

- Load balancing

  Definizione:

  Date m identiche macchine, n ≥ m jobs, job j ha processing time t_j.
  - Job j deve essere eseguito contiguamente su una macchina
  - Una macchina può processare al più un job alla volta
  - Assegnare ogni job a una macchina affinché il makespan sia minimo

  Def load:

  Sia S[i] il sottinsieme di job assegnati alla macchina i.

  Il load della macchina i è $L[i] = \Sigma_{j \in S[i]} t_j$.

  Def makespan:

  Il makespan è il valore massimo di load di ogni macchina $L = \max_i L[i]$

  List scheduling algo:
  - Considera i job in un qualche ordine fisso
  - Assegna il job j alla macchina i il cui carico è finora il più piccolo

LIST-SCHEDULING $(m, n, t_1, t_2, \ldots, t_n)$

FOR $i = 1$ TO m

$L[i] \leftarrow 0.$ ⟵ load on machine i

$S[i] \leftarrow \varnothing.$ ⟵ jobs assigned to machine i

FOR $j = 1$ TO n

$i \leftarrow \text{argmin}_k\, L[k].$ ⟵ machine i has smallest load

$S[i] \leftarrow S[i] \cup \{j\}.$ ⟵ assign job j to machine i

$L[i] \leftarrow L[i] + t_j.$ ⟵ update load of machine i

RETURN $S[1], S[2], \ldots, S[m].$

Complessità:

O(n log n)
Usando una coda con priorità per i carichi L[k]

Lemma 1. per tutti i k : il makespan ottimo $L^* \geq t_k$.

Lemma 2. il makespan ottimo $L^* \geq \frac{1}{m}\sum_k t_k$.

Teorema:

L'algoritmo greedy è di **2-approssimazione**

Dim:

- Cosidero il load L[i] della macchina i che crea il bottleneck
- Quando il job j viene assegnato alla macchina i, i ha il load minore.
- il suo carico prima dell'assegnazione è L[i] - t_j ;
- Hence L[i] - t_j ≤ L[k] per ogni 1 ≤ k ≤ m
- Sommo le disuguaglianze su tutti i k e divido per m:

$$
\begin{aligned}
L[i] - t_j &\leq \frac{1}{m}\sum_k L[k] \\
&= \frac{1}{m}\sum_k t_k \\
\text{Lemma 2} \longrightarrow \quad &\leq L^*.
\end{aligned}
$$

Now, $L = L[i] = \underbrace{(L[i] - t_j)}_{\leq L^*} + \underbrace{t_j}_{\leq L^*} \leq 2L^*$.

above inequality Lemma 1

- k-center problem 2-approximation

Definizione:

Dato un set di n siti $s_1,\ldots,s_n$ e un intero k > 0, Selezionare k centri C affinché la distanza tra un sito e il centro più vicino sia minima

Notazioni:

- dist(x,y) = distanza tra x e y
- dist(s_i, C) = $\min_{c \in C}$ dist(s_i, c) = distanza tra s_i e il centro più vicino
- r(C) = $\max_i$ dist(s_i, C) = raggio di copertura minima

  Goal. Trovare un set di centri che minimizzano r(C), con |C| = k.

Greedy algo:

Sceglie ripetutamente il sito più lontano da qualsiasi centro finora creato come prossimo centro

```
Greedy-Center-Selection(k, n, s1,s2,...,sn) {

   C = ϕ
   repeat k times {
      Select a site si with maximum dist(si, C)
      Add si to C
   }                          site farthest from any center
   return C
}
```

Teorema:

Sia C^* un insieme di centri ottimo. Allora $r(C) \leq 2r(C^*)$

Dim: *(per assurdo)*

- Assumiamo che $(C^*) < ½\, r(C)$.
- Per ogni sito c_i in C , si consideri una sfera di raggio ½ r(C) attorno ad esso
- Avremo esattamente un c_i^* in ogni sfera
- Ogni sfera con centro $c_i \in C$ deve cotenere un centro in C^* *(altrimenti $dist(c_i, C^*) \geq ½\, r(C) > r(C^*)$)*
- Le sfere sono disgiunte e $|C|=|C^*|$.
- Consideriamo un sito s e il suo centro più vicino c_i^* in C^*
- $Dist(s, C) \leq dist(s, c_i) \leq dist(s, c_i^*) + dist(c_i^*, c_i) \leq 2r(C^*)$
- Perciò $r(C) \leq 2r(C^*)$

- Hash table

  Definizione problema del dizionario:

Dato un universo U di possibili elementi, mantenere un sottoinsieme $S \subseteq U$ soggetti alle seguenti operazioni:

- **make-dictionary():** inizializzare un dizionario vuoto
- **insert(u):** aggiungere l'elemento $u \in U$ in s
- **delete(u):** cancella u da S, se u si trova in S
- **look-up(u)**: determinare se u è in S

Hash table:

- O(|S|) spazio
- O(1) tempo atteso ammortizzato